R language skills demonstration

2026-04-22

GitHub Documents

This is an R Markdown format used for publishing markdown documents to GitHub. When you click the **Knit** button all R code chunks are run and a markdown file (.md) suitable for publishing to GitHub is generated.

Content of this demonstration

1.Applied matrix algebra

2.Applied descriptive statistics analysis

3.Create informative graphs and comment on them

4.Do a regression analysis and comment on it

5.Compare distribution functions and comment on them. Use e.g. descriptive statistics, graphs etc and point out the difference between the distribution functions

6.Do a Monte Carlo Simulation (preferably based on empirical data) and comment on the results

7.Do several Monte Carlo Simulations with identical parameters and compare their outcomes (repectively their differences). Tip: Use the set.seed function to make your results reproducible.

8.Write a function to solve a problem in the fields of finance, risk management or asset management.

1.Applied matrix algebra

By using R code, matrix algebra problems can be solved, for example:

A <- matrix(c(1,2,0,2,3,1),3,2)

B <- matrix(c(1,-3,2,1,0,2),2,3)

Print out matrix A and B:

```
A <- matrix(c(1,2,0,2,3,1),3,2)
A
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    2    3
## [3,]    0    1
```

```
B <- matrix(c(1,-3,2,1,0,2),2,3)
B
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    0
## [2,]   -3    1    2
```

Matrix algebra: B multiply A

```
C <- B %*% A
C
```

```
##      [,1] [,2]
## [1,]    5    8
## [2,]   -1   -1
```

2.Applied descriptive statistics analysis

Example:

Consider the data for the change in personal disposable income in Austria (rounded values in %) from the table below.

And, calculate sample size n, number of attributes k and the absolute and relative frequencies. Use the data to construct the empirical CDF ˆF(x) in table form.

Year Change in the disposable income per capita in Austria (rounded values)

2001 2

2002 2

2003 4

2004 4

2005 7

2006 5

2007 5

2008 3

2009 0

2010 1

2011 3

2012 4

2013 0

2014 1

2015 0

2016 3

2017 2

2018 3

2019 3

2020 -2

```r
change <- c(2,2,4,4,7,5,5,3,0,1,3,4,0,1,0,3,2,3,3,-2)
cat("The the changes over years are (2001-2020):\n", change)
```

```
## The the changes over years are (2001-2020):
##  2 2 4 4 7 5 5 3 0 1 3 4 0 1 0 3 2 3 3 -2
```

```r
n <- length(change)
cat("The sample size n is:", n, "\n")
```

```
## The sample size n is: 20
```

```
k <- length(unique(change)) #calculate number of attributes k
cat("number of attributes k is:", k, "\n")
```

```
## number of attributes k is: 8
```

```
abs_freq <- table(change)
# The the absolute frequencies are:
abs_freq
```

```
## change
## -2  0  1  2  3  4  5  7
##  1  3  2  3  5  3  2  1
```

```
rel_freq <- prop.table(abs_freq)
# The the relative frequencies are:
rel_freq
```

```
## change
##   -2    0    1    2    3    4    5    7
## 0.05 0.15 0.10 0.15 0.25 0.15 0.10 0.05
```

```
ecdf_table <- cumsum(rel_freq)
# construct the Empirical CDF ^F(x)
# (which is ecdf - Empirical Cumulative Distribution Function) in table form:
ecdf_table
```

```
##   -2    0    1    2    3    4    5    7
## 0.05 0.20 0.30 0.45 0.70 0.85 0.95 1.00
```

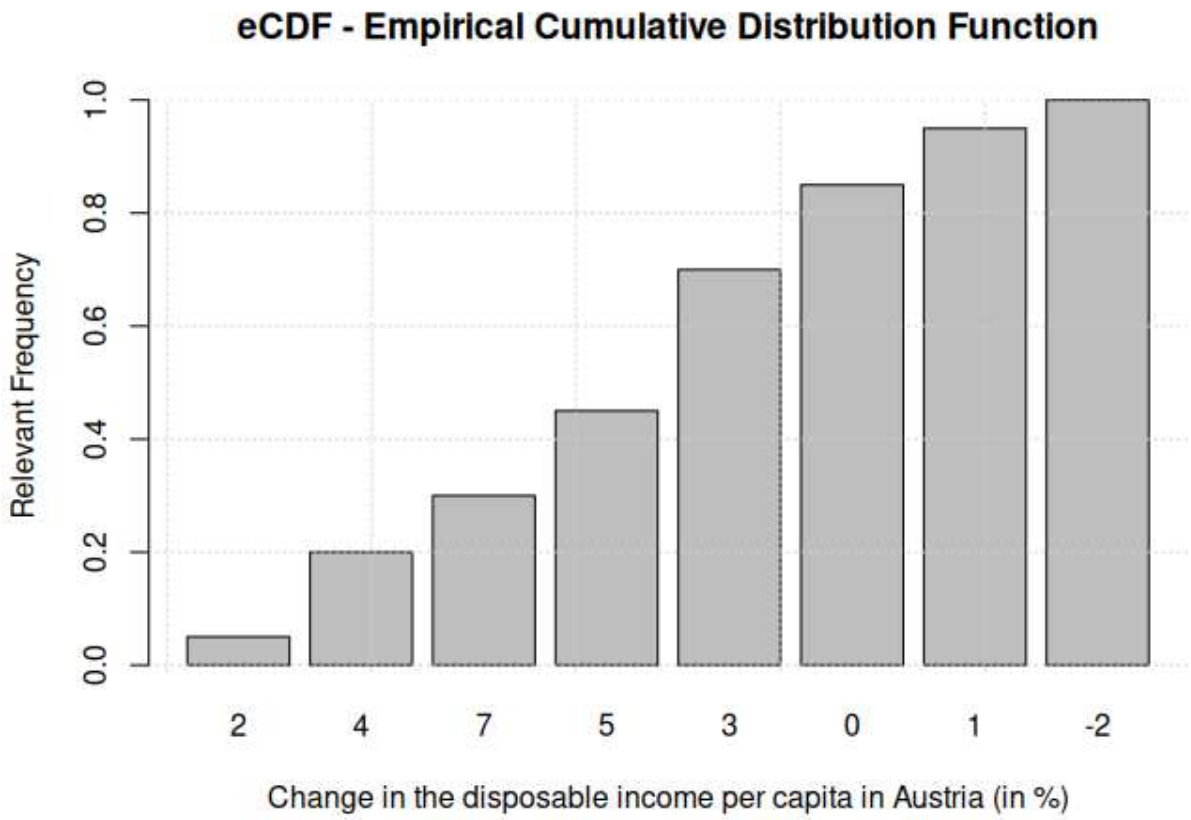
3.Create informative graph

Example:

Consider the data for the change in personal disposable income in Austria (rounded values in %) from the table in the previous sector, use them to construct the Empirical

CDF ˆF(x) as an informative graph.

```
change <- c(2,2,4,4,7,5,5,3,0,1,3,4,0,1,0,3,2,3,3,-2) #data
#x_vals <- sort(unique(change))
xaxis <- unique(change) #attributes of the changes (-2 to +7)
abs_freq <- table(change) #absolute frequencies
rel_freq <- prop.table(abs_freq) #relative frequencies
ecdf <- cumsum(rel_freq) # ecdf - Empirical Cumulative Distribution Function

barplot(ecdf,main="eCDF - Empirical Cumulative Distribution Function",
        xlab="Change in the disposable income per capita in Austria (in %)",
        ylab="Relevant Frequency",names.arg=xaxis)
grid()
```



4.Do a regression analysis

Example:

Linear Regression: Speed vs. Stopping Distance

The most classic example of regression analysis is predicting Stopping Distance of a running car based on Speed of the car. (On the earth that we're living on, probably, as a car goes faster, it takes longer to stop.)

We want to build a mathematical model to answer: “If I increase my speed by 1 m/h, how many extra distance will it take to stop?” In statistics, we use the Simple Linear Regression formula:

Stopping distance (dist) = The change in distance for every 1 unit of speed * Speed.

In this analysis, we model the relationship between a car’s speed and the distance required to come to a complete stop.

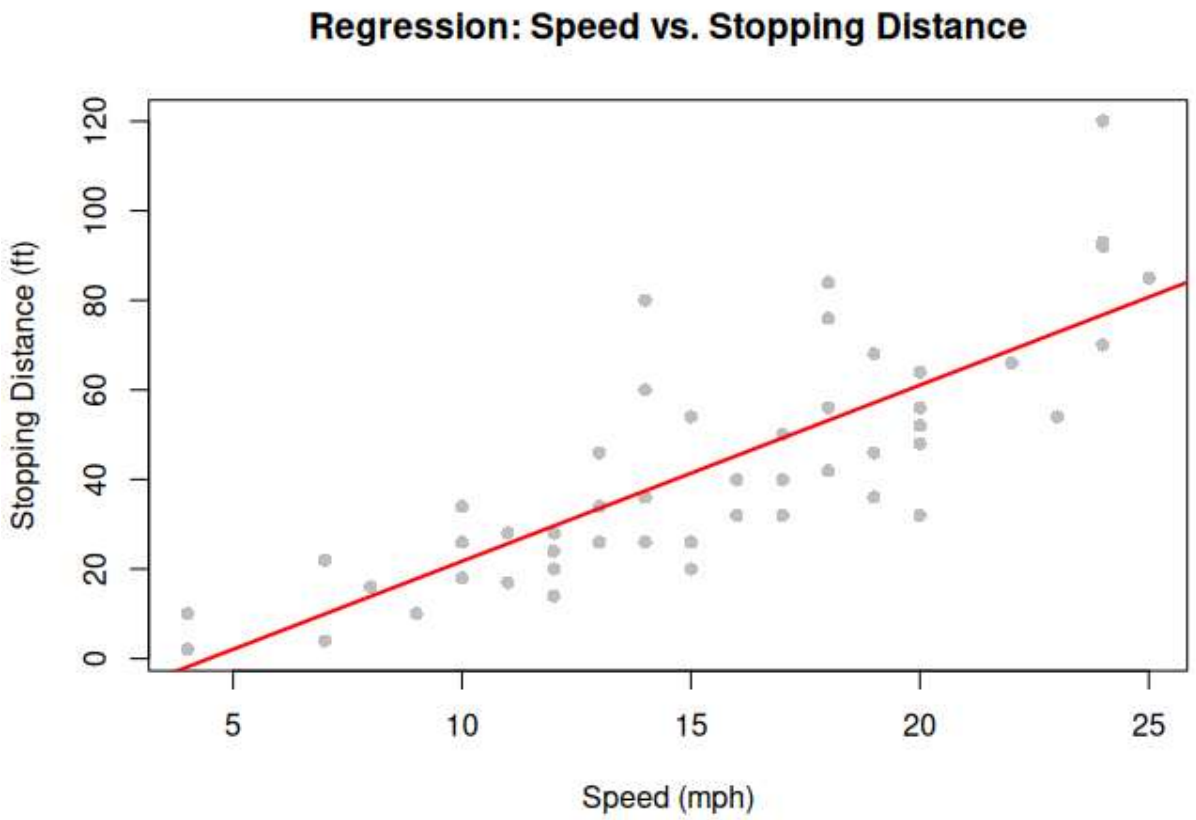
A built-in dataset of R called ‘cars’ that recorded exactly this analysis in the 1920s is used in the code below.

```
# 1. Load the data
data(cars)

# 2. Fit the Linear Model (lm)
# Formula: distance is a function of speed (dist ~ speed)
# The tilde ~ means "is modeled by"
model <- lm(dist ~ speed, data = cars)

# 3. Display the statistical results
summary(model)
```

```
##
## Call:
## lm(formula = dist ~ speed, data = cars)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -29.069  -9.525  -2.272   9.215  43.201
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -17.5791     6.7584  -2.601   0.0123 *
## speed         3.9324     0.4155   9.464 1.49e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.38 on 48 degrees of freedom
## Multiple R-squared:  0.6511, Adjusted R-squared:  0.6438
## F-statistic: 89.57 on 1 and 48 DF,  p-value: 1.49e-12
```

```
# 4. Visualize the results
plot(cars$speed, cars$dist,
     main = "Regression: Speed vs. Stopping Distance",
     xlab = "Speed (mph)",
     ylab = "Stopping Distance (ft)",
```

```
     pch = 16, col = "gray")

# Add the regression line
# 'abline(model)' automatically draws the "Line of Best Fit"
# onto the plot based on the regression results.
abline(model, col = "red", lwd = 2)
```



So forth, our regression analysis on the linear example of Speed vs. Stopping Distance is done.

5.Compare distribution functions

Compare distribution functions and comment on them. Use e.g. descriptive statistics, graphs etc.

Example:

There are two groups of data which have very distinct distributions - the oetal length of two kinds of Iris flower species.

Comparison Strategy:

To compare distributions effectively, we look at three things.

1.Location (Center): Where is the bulk of the data located? (Mean/Median)

2.Scale (Spread): How “stretched out” is the data? (Standard Deviation/IQR)

3.Shape: Is it skewed, or does one “stochastically dominate” the other (is one curve always to the right of the other)?

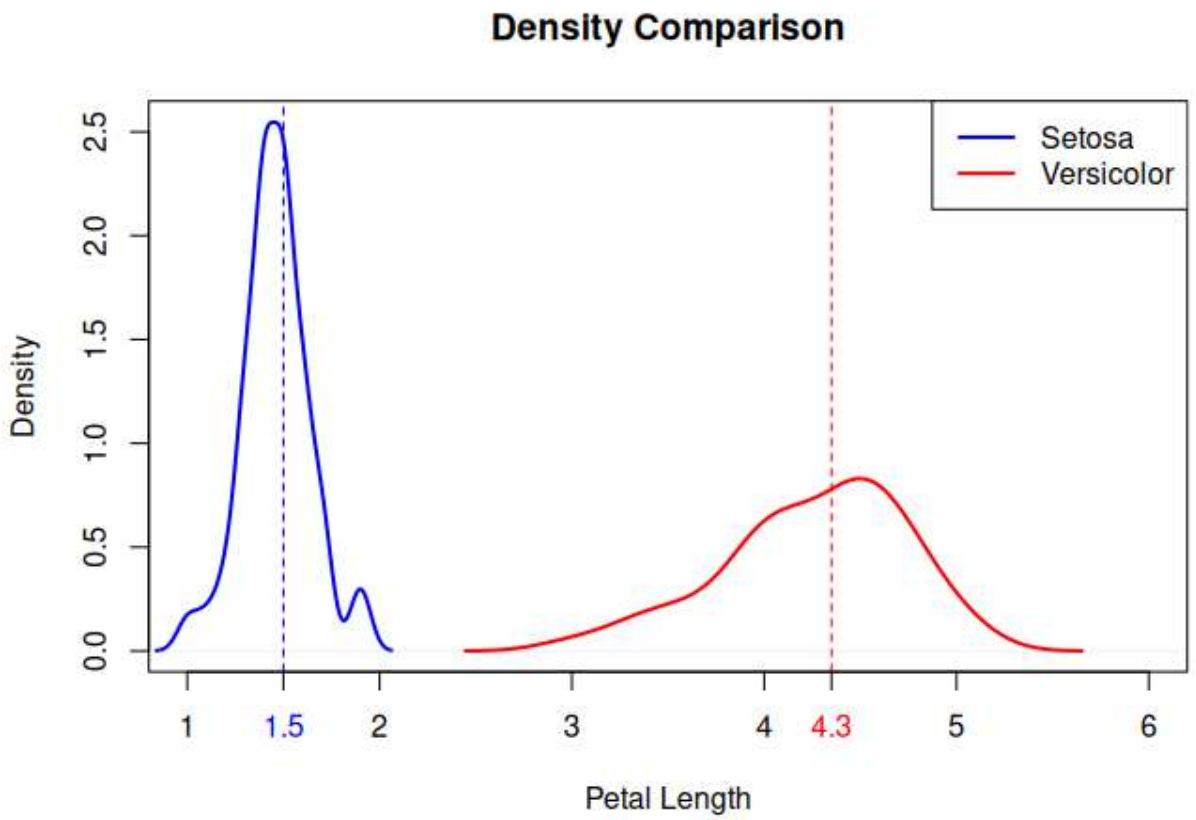
```
# 1. Descriptive Statistics Comparison
summary(iris)
```

```
##   Sepal.Length    Sepal.Width     Petal.Length    Petal.Width   
##  Min.   :4.300   Min.   :2.000   Min.   :1.000   Min.   :0.100  
##  1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600   1st Qu.:0.300  
##  Median :5.800   Median :3.000   Median :4.350   Median :1.300  
##  Mean   :5.843   Mean   :3.057   Mean   :3.758   Mean   :1.199  
##  3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100   3rd Qu.:1.800  
##  Max.   :7.900   Max.   :4.400   Max.   :6.900   Max.   :2.500  
##        Species  
##  setosa    :50  
##  versicolor:50  
##  virginica :50  
##                 
##                 
##
```

```
# We first split the data into two groups and calculate their summary statistics.
# Subset the data
setosa <- iris$Petal.Length[iris$Species == "setosa"]
versicolor <- iris$Petal.Length[iris$Species == "versicolor"]
#virginica <- iris$Petal.Length[iris$Species == "virginica"]

# Create a comparison table
stats_comp <- data.frame(
  Metric = c("Mean", "Median", "Std Dev", "Min", "Max"),
  Setosa = c(mean(setosa), median(setosa), sd(setosa), min(setosa), max(setosa)),
  Versicolor = c(mean(versicolor), median(versicolor), sd(versicolor), min(versico
#  ,virginica = c(mean(virginica), median(virginica), sd(virginica), min(virginica
)

print(stats_comp)
```

```
##    Metric   Setosa Versicolor
## 1    Mean 1.462000   4.260000
## 2  Median 1.500000   4.350000
## 3 Std Dev 0.173664   0.469911
## 4     Min 1.000000   3.000000
## 5     Max 1.900000   5.100000
```

```r
# 2. Visualizing PDF (Density Plots)

# Plot the first density
plot(density(setosa), col = "blue", lwd = 2,
     main = "Density Comparison",
     xlab = "Petal Length", xlim = c(1, 6))

# Add the second density
lines(density(versicolor), col = "red", lwd = 2)

# Add vertical lines at the median positions
med_setosa <- median(setosa)
med_versicolor <- median(versicolor)

abline(v = c(med_setosa,med_versicolor),
       col = c("blue","red"), lty = 2)

# Mark the corresponding value on the x-axis of the vertical lines
axis(side = 1, at = med_setosa,
     labels = round(med_setosa, 1), col.axis = "blue", las = 1)
axis(side = 1, at = med_versicolor,
     labels = round(med_versicolor, 1), col.axis = "red", las = 1)

legend("topright", legend = c("Setosa", "Versicolor"),
       col = c("blue", "red"),
       lwd = 2)
```



To compare distributions effectively, we look at:

1.Location (Center): (Mean/Median)

The center of setosa-data is apparently way smaller than it of versicolor-data.

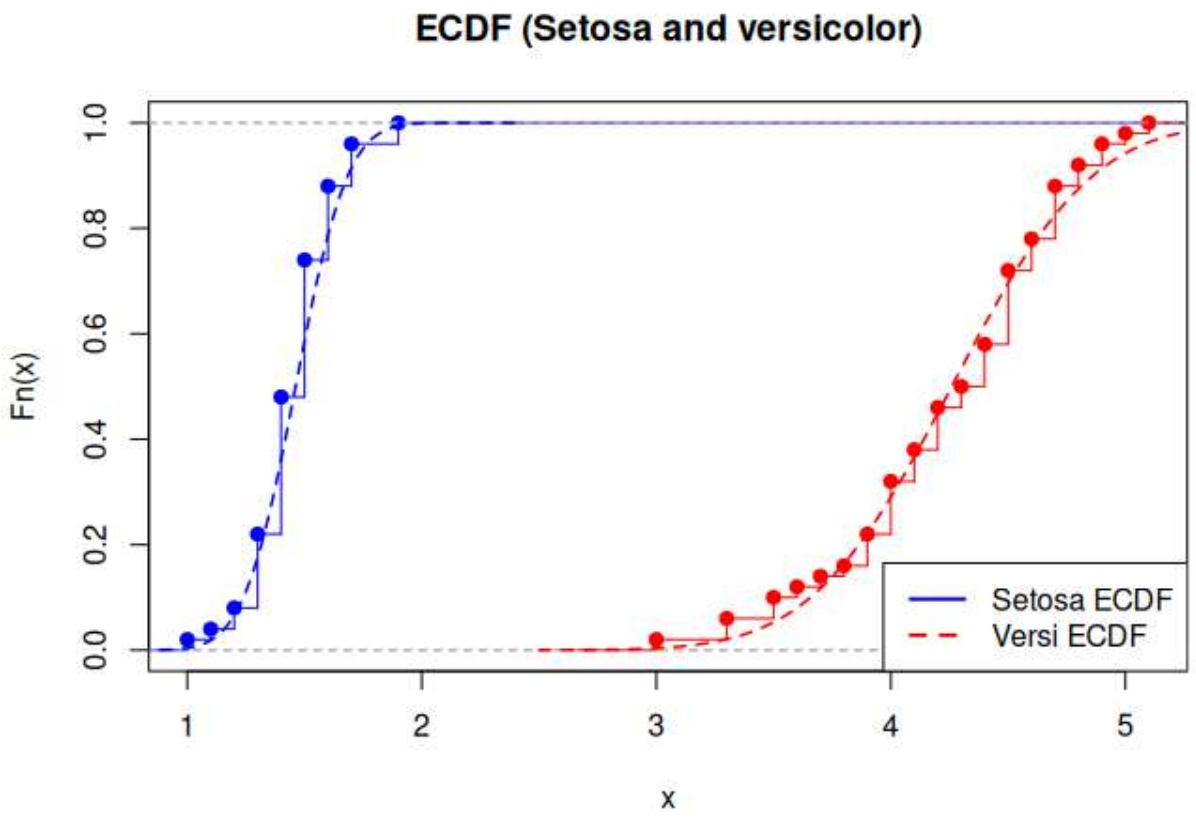
2.Scale (Spread): How “stretched out” is the data? (Standard Deviation)

The setosa-data is less stretched out than the versicolor-data (standard deviation is smaller).

3.Shape: Is it skewed, or does one “stochastically dominate” the other (is one curve always to the right of the other)?

Both data sets are not apparently skwed. The versicolor-data curve is always to the right of the other.

```
# Comparing ECDF (Empirical Cumulative Distribution Functions)

# Prepare for Setosa
mu_s <- mean(setosa)
sd_s <- sd(setosa)
# Create a sequence of values for the smooth line
x_seq_s <- seq(min(setosa) - 0.5, max(setosa) + 0.5, length.out = 100)
# Calculate the theoretical normal CDF
prob_s <- pnorm(x_seq_s, mean = mu_s, sd = sd_s)

# Prepare for Versicolor
mu_v <- mean(versicolor)
sd_v <- sd(versicolor)
x_seq_v <- seq(min(versicolor) - 0.5, max(versicolor) + 0.5, length.out = 100)
prob_v <- pnorm(x_seq_v, mean = mu_v, sd = sd_v)

# Plotting
plot(ecdf(setosa),
     col = "blue", verticals = TRUE, do.points = TRUE,
     xlim = range(setosa, versicolor),
     main = "ECDF (Setosa and versicolor)")


# Add Versicolor
plot(ecdf(versicolor),
     col = "red", verticals = TRUE, do.points = TRUE,
     add = TRUE)

# Add the 'Symmetry/Normal' Approximation line for Setosa
lines(x_seq_s, prob_s, col = "blue", lty = 2, lwd = 1.5)
# Add the 'Symmetry/Normal' Approximation line for Versicolor
lines(x_seq_v, prob_v, col = "red", lty = 2, lwd = 1.5)

legend("bottomright", legend = c("Setosa ECDF", "Versi ECDF"),
       col = c("blue",  "red"), lty = c(1, 2), lwd = 2)
```



The differences we can point out comparing these two (Empirical Cumulative) distribution functions are:

1. Location (The “Shift”)

The ECDF of Versicolor is always to the right of Setosa (Versicolor stochastically dominates Setosa).

2. Scale or Dispersion (The “Steepness”)

This tells you about the variance or “spread” of the data. Setosa-data has a steeper curve: it has lower variance.

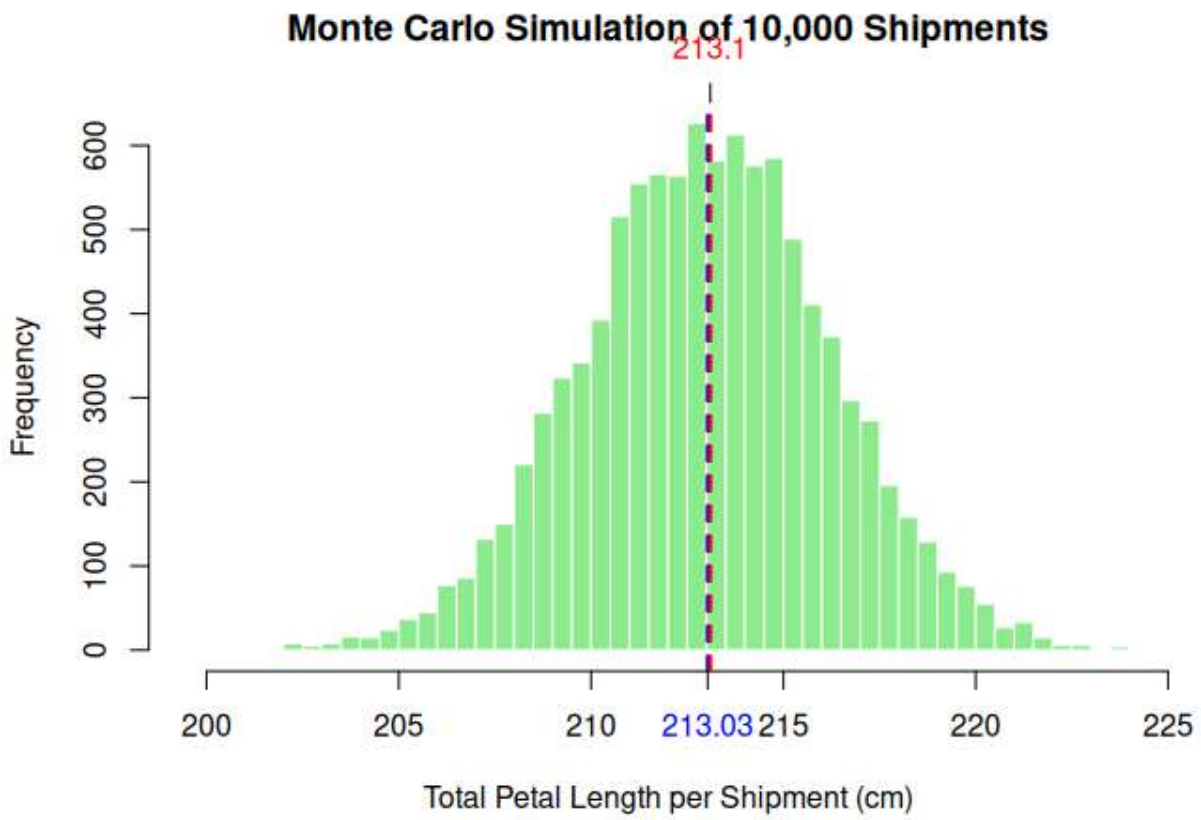
6. Do a Monte Carlo Simulation

To do a Monte Carlo Simulation based on empirical data (Iris flowers, in this case), we are essentially asking R-code to “pretend” it is running an experiment thousands of times, by randomly sampling from the existing data (usually, a limited amount).

For this example, let’s simulate the expected total weight of a shipment of 50 Iris flowers (specifically the Versicolor species).

Since we only have 50 real measurements in the dataset, we will simulate 10,000 “virtual shipments” to see the range of possible outcomes.

```r
# 1. Prepare Empirical Data
versicolor_lengths <- iris$Petal.Length[iris$Species == "versicolor"]

# 2. Set Simulation Parameters
n_flowers <- 50          # Number of flowers per shipment
n_simulations <- 10000   # Number of virtual shipments to run
set.seed(123)            # Ensures results are reproducible

# 3. Run the Simulation
# We use replicate() to repeat the sampling process 10,000 times
results <- replicate(n_simulations, {
  # Randomly pick 50 flowers from our empirical data (with replacement)
  shipment <- sample(versicolor_lengths, size = n_flowers, replace = TRUE)
  sum(shipment) # Calculate total length for this virtual shipment
})

# 4. Analyze Results
mean_total <- mean(results)
median <- median(results)

# 5. Visualization
hist(results,
     breaks = 50,
     col = "lightgreen", border = "white",
     main = "Monte Carlo Simulation of 10,000 Shipments",
     xlab = "Total Petal Length per Shipment (cm)")

# Mark the corresponding value on the x-axis of the vertical lines
abline(v = mean_total, col = "blue", lwd = 2, lty = 2)
axis(side = 1, at = mean_total,
     labels = round(mean_total, 2), col.axis = "blue", las = 1)

abline(v = median, col = "red", lwd = 2, lty = 2)
axis(side = 3, at = median,
     labels = round(median, 2), col.axis = "red", las = 1)
```




```
# mean of the total 10000 times of the simulation is in blue down on the x-axis
# median of the total 10000 times of the simulation is in red color up there
```

7.Do several Monte Carlo Simulations

Requirments:

Do several Monte Carlo Simulations with identical parameters (like in the previous chapter) and compare their outcomes (respectively their differences). Tip: Use the set.seed function to make your results reproducible.

Setting-up:

To simulate the expected total weight of a shipment of 50 Iris flowers (specifically the Versicolor species), we simulate 1,5,100 and (like in the previous chapter) 10,000 times of “virtual shipments” to see the range of possible outcomes (Since we only have 50 real measurements in the dataset, after all).

After running the simulation of 1, 5 and 100 times of “virtual shipments”, we can overlay the density of the different simulations to see if they overlap and to compare them.

```r
# 1. simulate “virtual shipments” 1 time

versicolor_lengths <- iris$Petal.Length[iris$Species == "versicolor"]
                          # Prepare Empirical Data
n_flowers <- 50           # Number of flowers per shipment
n_simulations_1 <- 1    # Number of virtual shipments to run
set.seed(123)             # Ensures results are reproducible

# We use replicate() to repeat the sampling process 1 times
results_1 <- replicate(n_simulations_1, {
  shipment <- sample(versicolor_lengths, size = n_flowers, replace = TRUE)
  # Randomly pick 50 flowers from our empirical data (with replacement)
  sum(shipment) # Calculate total length for this virtual shipment
})

# The expected total weight of a shipment of 50 Iris flowers
# simulated after 1 time of Monte Carlo simulation is
results_1
```

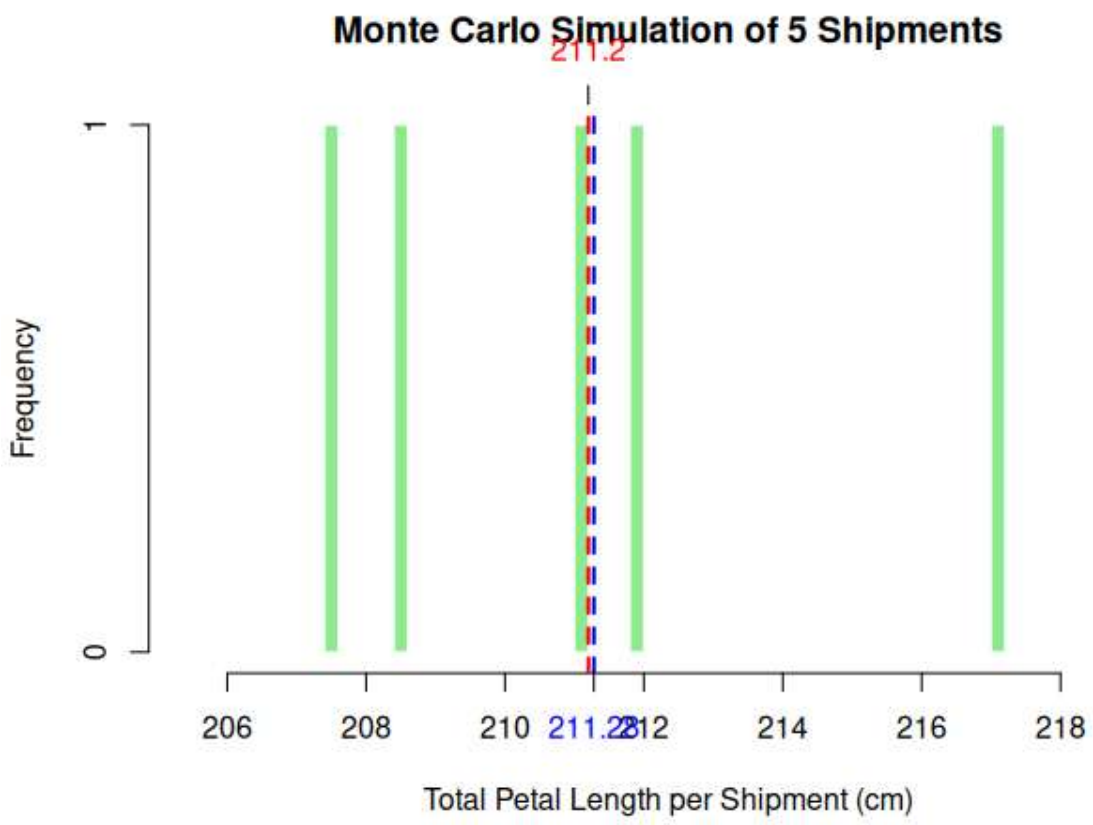
```
## [1] 219.7
```

```r
# 2. simulate “virtual shipments” 5 time
n_simulations_5 <- 5    # Number of virtual shipments to run
results_5 <- replicate(n_simulations_5, {
  shipment <- sample(versicolor_lengths, size = n_flowers, replace = TRUE)
  # Randomly pick 50 flowers from our empirical data (with replacement)
  sum(shipment) # Calculate total length for this virtual shipment
})

# The expected total weight of a shipment of 50 Iris flowers
# simulated after 5 time of Monte Carlo simulation is
results_5
```

```
## [1] 208.6 211.2 207.5 217.1 212.0
```

```r
# Analyze Results
mean_total <- mean(results_5)
median <- median(results_5)

# Visualization
r <- range(results_5) # design the length of x-axis
my_xlim <- c(r[1] * 0.99, r[2] * 1.01) # add 1% more to the range


hist(results_5,
     breaks = 50,
```

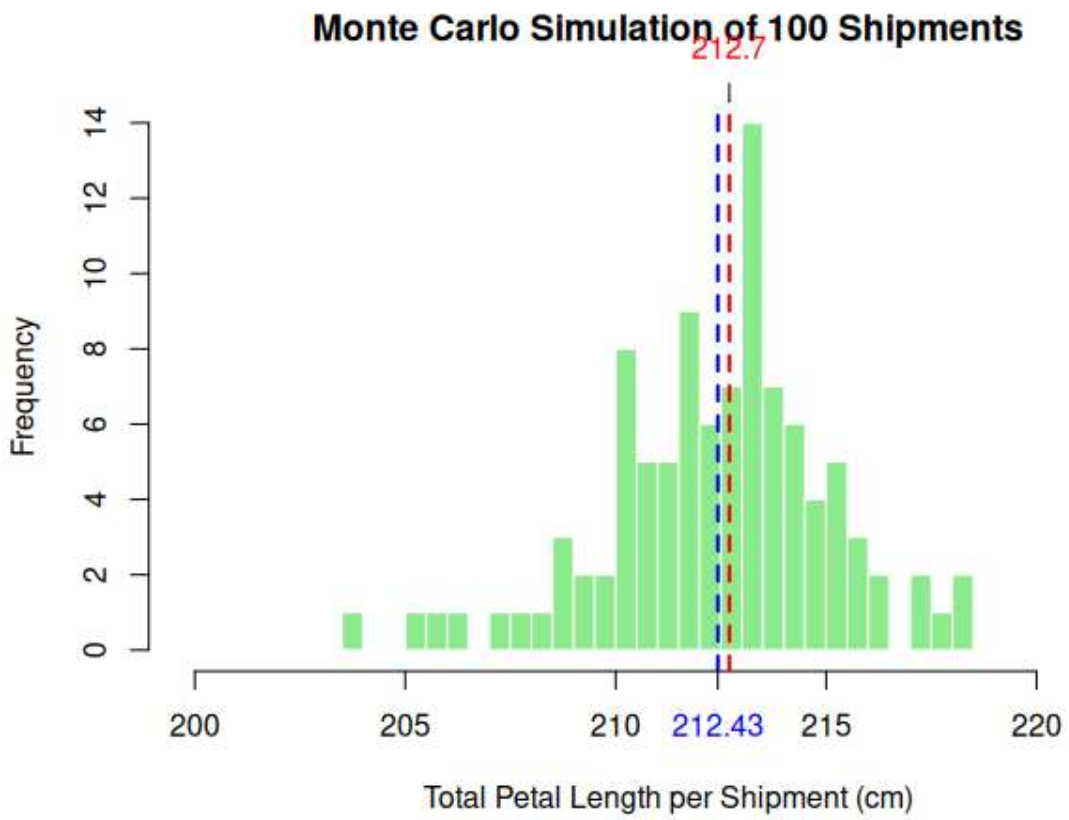
```
     xlim = my_xlim,
     col = "lightgreen", border = "white",
     main = "Monte Carlo Simulation of 5 Shipments",
     xlab = "Total Petal Length per Shipment (cm)")

# Mark the corresponding value on the x-axis of the vertical lines
abline(v = mean_total, col = "blue", lwd = 2, lty = 2)
axis(side = 1, at = mean_total,
     labels = round(mean_total, 2), col.axis = "blue", las = 1)

abline(v = median, col = "red", lwd = 2, lty = 2)
axis(side = 3, at = median,
     labels = round(median, 2), col.axis = "red", las = 1)
```




```
# 3. simulate “virtual shipments” 100 time
n_simulations_100 <- 100   # Number of virtual shipments to run
results_100 <- replicate(n_simulations_100, {
  shipment <- sample(versicolor_lengths, size = n_flowers, replace = TRUE)
  # Randomly pick 50 flowers from our empirical data (with replacement)
  sum(shipment) # Calculate total length for this virtual shipment
})

# The expected total weight of a shipment of 50 Iris flowers
# simulated after 100 time of Monte Carlo simulation is
results_100
```

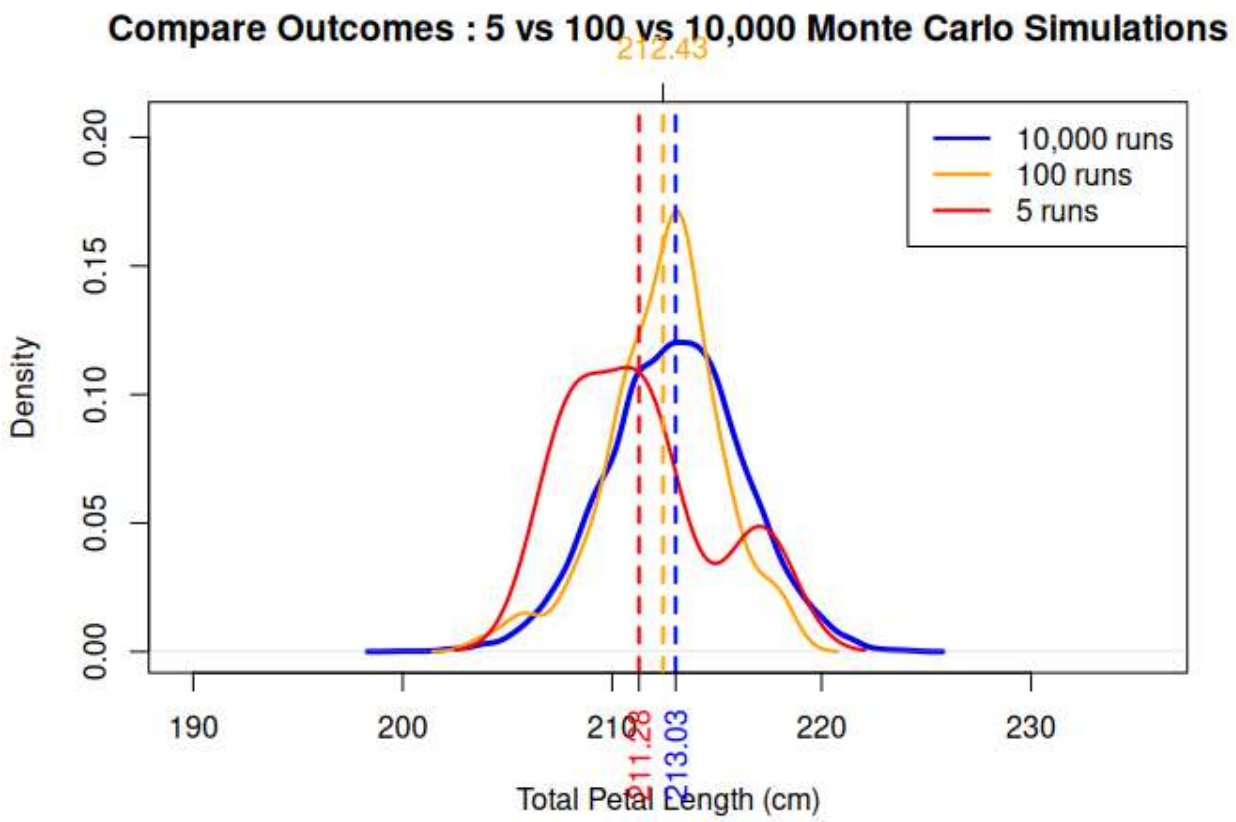
```
##   [1] 217.5 205.4 210.3 208.7 213.2 211.5 211.6 212.6 210.6 209.7 213.6 215.5
##  [13] 212.2 212.7 210.3 215.0 214.1 213.7 209.1 210.8 211.1 215.8 212.9 212.4
##  [25] 213.3 211.6 210.5 213.4 208.9 213.8 213.3 215.1 213.3 211.7 210.4 213.6
##  [37] 212.4 214.9 212.0 211.9 214.5 216.5 207.8 213.6 208.7 209.7 213.1 210.1
##  [49] 214.6 209.1 212.5 211.2 203.9 211.1 213.4 213.1 210.8 213.6 215.8 215.9
##  [61] 213.2 216.5 210.3 212.5 212.0 212.4 218.3 213.1 212.9 215.1 211.1 214.3
##  [73] 210.1 212.8 217.7 211.8 218.2 210.8 215.3 205.7 211.7 206.1 213.7 214.5
##  [85] 212.7 214.3 213.5 214.2 211.6 214.8 210.4 213.3 210.8 213.2 215.2 217.4
##  [97] 213.5 212.7 208.1 207.5
```

```r
# Analyze Results
mean_total <- mean(results_100)
median <- median(results_100)

# Visualization
r <- range(results_100) # design the length of x-axis
my_xlim <- c(r[1] * 0.98, r[2] * 1.02) # add 2% more to the range

hist(results_100,
     breaks = 50,
     xlim = my_xlim,
     col = "lightgreen", border = "white",
     main = "Monte Carlo Simulation of 100 Shipments",
     xlab = "Total Petal Length per Shipment (cm)")

# Mark the corresponding value on the x-axis of the vertical lines
abline(v = mean_total, col = "blue", lwd = 2, lty = 2)
axis(side = 1, at = mean_total,
     labels = round(mean_total, 2), col.axis = "blue", las = 1)

abline(v = median, col = "red", lwd = 2, lty = 2)
axis(side = 3, at = median,
     labels = round(median, 2), col.axis = "red", las = 1)
```




```
# 4. Overlay the density of the different simulations (1,5,100, 10000 times)

# density doesn't work well for 1 point, so we focus on 5+
# We've had: 'results_5', 'results_100'
# and 'results' (10,000 from the previous chapter)

# Calculate density objects
d5    <- density(results_5)
d100  <- density(results_100)
d10k  <- density(results)

# Create the canvas using the 10k results (it has the most reliable range)
plot(d10k, col = "blue", lwd = 3,
     main = "Compare Outcomes : 5 vs 100 vs 10,000 Monte Carlo Simulations",
     xlab = "Total Petal Length (cm)",
     xlim = range(results) * c(0.95, 1.05), # Add 5% padding
     ylim = c(0, max(d100$y) * 1.2))          # Add 20% more space for the legend

# Overlay the others
lines(d100,  col = "orange", lwd = 2)
lines(d5,    col = "red", lwd = 2)

# Add a legend to identify the lines
legend("topright",
       legend = c("10,000 runs", "100 runs", "5 runs"),
       col = c("blue", "orange", "red"),
       lty = c(1, 1, 1, 3), lwd = 2)
```

```
# Mark the corresponding mean value on the x-axis of the vertical lines
abline(v = mean(results), col = "blue", lwd = 2, lty = 2)
axis(side = 1, at = mean(results), labels = round(mean(results), 2),
     col.axis = "blue", las = 2)

abline(v = mean(results_100), col = "orange", lwd = 2, lty = 2)
axis(side = 3, at = mean(results_100), labels = round(mean(results_100), 2),
     col.axis = "orange", las = 1)

abline(v = mean(results_5), col = "red", lwd = 2, lty = 2)
axis(side = 1, at = mean(results_5), labels = round(mean(results_5), 2),
     col.axis = "red", las = 2)
```



After we've done one Monte Carlo simulation in the previous chapter (6), doing several of them in this chapter allows us to see how randomness behaves.

8.Write a function

Write a function to solve a problem in the fields of finance, risk management or asset management.

Designed problem in the fields: Calculating Value at Risk (VaR)

VaR answers a very simple but critical question: "What is the maximum amount I could lose on this investment over a specific time period with a certain level of confidence?"

1. The Scenario

You are a Risk Manager at an investment firm. You have a portfolio worth EUR 12,000. You want to know the “Worst Case Scenario” loss for a single day, with 95% confidence, based on historical volatility.

2. We will write a custom function called

‘calculate_var’.

```
# Custom Function for Value at Risk (VaR)

# Parameters:
#   investment: The total dollar amount invested
#   mean_ret: The average daily return (historical)
#   sd_ret: The standard deviation of daily returns (volatility)
#   conf_level: The confidence level (default is 0.95)

calculate_var <- function(investment, mean_ret, sd_ret, conf_level = 0.95) {

  # Step 1: Find the Z-score for the lower tail of the normal distribution
  # For 95% confidence, we look at the 5th percentile (1 - 0.95)
  # -the value below which 5% of the data falls and above which 95% of the data li
  z_score <- qnorm(1 - conf_level)

  # Step 2: Calculate the percentage return at that threshold
  threshold_return <- mean_ret + (z_score * sd_ret)

  # Step 3: Convert that percentage loss into a dollar amount
  # We take the absolute value because VaR is usually expressed as a positive loss
  var_amount <- abs(investment * threshold_return)

  return(var_amount)
}

# --- Application Example ---

# Let's say:
# Portfolio = EUR 12,000
# Avg Daily Return = 0.01 (1%)
# Daily Volatility (SD) = 0.02 (2%)

my_risk <- calculate_var(investment = 12000, mean_ret = 0.01, sd_ret = 0.02)

cat("The 1-day 95% Value at Risk (VaR) is: EUR", round(my_risk, 2))
```

```
## The 1-day 95% Value at Risk (VaR) is: EUR 274.76
```

3. Interpretation of the result

The result indicates that with 95% confidence, in a worst case scenario, the loss per day (based on an assumed historical volatility of ‘Avg Daily Return is 1%, and Daily Volatility (SD) = 0.02’) is VaR amount of money (when the portfolio is 12,000 EUR).

So to say, there is only a 5% chance that your invested portfolio will lose more than this amount in a single day.

VaR calculation and controlling is helpful under the mindset of

‘limits consideration’,

‘max loss bearing’

and ‘prepare for the worst, hope for the best’.

Contact

Jane Jingyi LU

LinkedIn: www.linkedin.com/in/jane-jingyi-lu